

# remote-operations

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

이 문서는 remote-operations 문서의 시작점이다. **설계**는 부팅 orchestration의 책임 범위와 각 스크립트의 역할을 설명하고, **운영**은 실제 서버 관리자가 배포·점검·장애 대응에 사용하는 명령을 설명하며, **설정**은 config 파일을 어떻게 준비하는지 설명한다.

## 문서 구성

| 문서     | 핵심 내용                                  |
|--------|----------------------------------------|
| 현재 페이지 | 책임 범위와 문서 안내                           |
| 설계     | 디렉터리 지도, 부팅 상태 머신, 핵심 기능, 설계상 지켜야 할 경계 |
| 운영     | 사용법, dry-run과 검증, 실패 처리 원칙, 새 서버 추가    |
| 설정     | config 파일 준비와 설정 모델                    |

처음 보는 사람은 **설계 문서**에서 전체 구조를 먼저 확인하고, 실제 서버를 켜거나 점검해야 할 때 **운영 문서**로, config 파일을 새로 준비해야 할 때 **설정 문서**로 이동한다.

전체 서버 동시 부팅 전환과 Slack 알림 백엔드 큐 연동 검증은 완료되어 이 문서들에 반영됨. `.sh` → `.py` 전환 작업은 아직 진행 전이며, 반영되면 이 페이지와 설계/운영 문서 내용도 함께 갱신해야 한다.

# remote-operations 설계

개요 · 운영 · 설정

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

## 1. 책임 범위

`remote-operations`는 management host의 `remote-boot.service`가 실행하는 부팅 orchestration이다. Wake-on-LAN, 전체 대상 host health gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 `monitoring`에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 `remote-boot.service` 하나다.

## 2. 디렉터리 지도

| 경로                                                   | 핵심 기능                                       |
|------------------------------------------------------|---------------------------------------------|
| <code>script/run_remote_boot.sh</code>               | 전체 부팅 흐름의 entry point                       |
| <code>script/wake_targets.sh</code>                  | target 선택과 WOL magic packet 전송              |
| <code>script/wait_for_priority_servers.sh</code>     | 선택된 대상 전체의 host health gate                 |
| <code>script/check_server_boot_health.sh</code>      | host SSH 도달성, mount, host GPU 점검            |
| <code>script/restart_all_remote_containers.sh</code> | 기존 container 시작과 SSH/GPU post-check         |
| <code>script/create_test_container.sh</code>         | (부팅 흐름에서 호출되지 않는 독립 도구) 임시 GPU container 생성 |
| <code>script/delete_test_container.sh</code>         | (부팅 흐름에서 호출되지 않는 독립 도구) 임시 container 정리     |
| <code>script/common.sh</code>                        | config, target, Ansible, log, alert 공통 함수   |
| <code>script/install_remote_boot_service.sh</code>   | systemd unit 설치·활성화                         |
| <code>test/dry_run_remote_boot.sh</code>             | WOL/health/container/full flow simulation   |
| <code>test/integration_smoke_test.sh</code>          | 수동 Ansible/Docker/GPU 통합 확인                 |
| <code>test/test_slack_notification.sh</code>         | Slack webhook 설정과 알림 발송 테스트                 |
| <code>config/remote_boot.example.env</code>          | 공개 가능한 설정 구조                                |
| <code>config/remote_boot.local.env</code>            | 실제 MAC·webhook 등을 담는 ignored 설정             |

### 3. 부팅 상태 머신

```
pre-delay
|
v
wake all selected targets (동시)
|
v
health gate for all selected targets -- failure --> stop + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

과거에는 LAB1 / FARM1 에 있는 사용자 관리 DB(사용자, UID/GID, 할당 port와 사용자 container record를 관리하는 기준 데이터)를 먼저 기동하기 위해 LAB1 / FARM1 을 priority target으로 먼저 깨우고 health gate를 통과시킨 뒤 나머지 서버를 기동하는 2단계 구조였다. 이 priority/remaining 분리 로직은 제거되었고, 지금은 선택된 대상을 항상 동시에 wake + gate한다.

### 4. 핵심 기능

#### 4.1 target 정규화

설정은 FARM/LAB 전체 목록과 기본 대상을 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. all, domain group, 개별 FARM1 / LAB10 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

#### 4.2 1회 host health check

check\_server\_boot\_health.sh 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount
- host nvidia-smi

과거에는 이 뒤에 create\_test\_container.sh 로 임시 GPU container를 만들어 container 내부의 GPU/SSH까지 확인하고 delete\_test\_container.sh 로 정리하는 단계가 있었다. 여러 서버가 동시에 부팅되면 이 단계가 서버마다 GPU 전체와 메모리 192g를 잡고 무거운 이미지를 pull하는 부담으로 이어져 제거했다. 이제 host 레벨 확인까지만 하고,

container GPU 연동이 실제로 살아있는지는 `restart_all_remote_containers.sh` 가 기존 사용자 container를 대상으로 재확인한다 (§ 4.3). `create_test_container.sh` / `delete_test_container.sh` 자체는 삭제하지 않았으므로, 필요하면 독립적으로 실행할 수 있다.

### 4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle` 에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

### 4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh` 는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

## 5. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.
- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.

## remote-operations 운영

개요 · 설계 · 설정

### 1. 사용법

target 확인과 수동 WOL ( `admin_infra_server/remote-operations` 에서 실행):

```
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh  
sudo systemctl start remote-boot.service  
systemctl status remote-boot.service  
journalctl -u remote-boot.service -b
```

## 2. dry-run과 검증

```
./test/dry_run_remote_boot.sh wake FARM1 LAB1  
./test/dry_run_remote_boot.sh health FARM1  
./test/dry_run_remote_boot.sh containers FARM1  
./test/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./test/integration_smoke_test.sh
```

Slack 알림 설정을 확인할 때:

```
./test/test_slack_notification.sh --server-id FARM1
```

`REMOTE_BOOT_SLACK_ENABLED` 와 webhook URL이 설정돼 있어야 하며, 테스트 메시지를 실제로 전송해 alert 경로가 살아있는지 확인한다.

운영 전에는 최소한 다음을 확인한다.

- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- gate timeout이 선택된 서버 전체가 동시에 부팅되는 시간을 감안해도 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

### 3. 실패 처리 원칙

- 선택된 대상 중 gate를 통과하지 못한 서버가 있으면 container 재시작 단계로 넘어가지 않는 것이 기본이다.
- gate 단계에서 서버별 통과/대기 상태를 구분해 기록한다 (`wait_for_priority_servers.sh`가 통과한 서버와 재시도 중인 서버를 분리해서 로그로 남긴다).
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 `monitoring`으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

### 4. 새 서버 추가

1. `user-lifecycle/server_info/servers.jsonl`과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.
6. container restart를 한 서버에 제한해 검증한다.
7. `monitoring`의 scrape target/exporter 배포를 별도로 완료한다.

## remote-operations 설정

개요 · 설계 · 운영

### 설정 모델

설정은 `remote_boot.example.env`를 복사하여 local 파일에서 관리한다 (`admin_infra_server/remote-operations`에서 실행).

```
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

| 그룹        | 예                                        |
|-----------|------------------------------------------|
| target    | FARM/LAB 목록, 기본 target                   |
| 순서/gate   | pre-delay, gate timeout/poll             |
| container | restart enable, timeout, post-check poll |

| 그룹                      | 예                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| network                 | Ansible inventory, broadcast IP, MAC address                                                                                                     |
| health                  | 필수 mount와 host share template                                                                                                                    |
| test container (독립 도구용) | <code>create_test_container.sh</code> / <code>delete_test_container.sh</code> 가 쓰는 image, UID/GID, mount, memory, runtime — 부팅 흐름에서는 더 이상 쓰이지 않음 |
| logging/alert           | log path, rotate count, notify API와 webhook                                                                                                      |

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

## 첫 설정 체크리스트

이 저장소를 처음 pull한 뒤 dry-run 검증까지 마치는 순서:

1. 개인 SSH 공개키를 각 FARM/LAB 서버의 `~/.ssh/authorized_keys`에 등록
2. `~/ansible/inventory.ini` 준비 (`ansible_user`를 자신의 계정으로)
3. `remote_boot.example.env` → `remote_boot.local.env` 복사
4. `REMOTE_BOOT_ANSIBLE_INVENTORY`를 자신의 inventory 경로로 설정
5. `REMOTE_BOOT_MAC_*` 값을 실제 MAC으로 채움
6. `REMOTE_BOOT_SLACK_ENABLED=true`와 `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 설정
7. 운영 문서의 dry-run 명령으로 검증

## 개인 Ansible inventory 준비

관리 데스크탑은 여러 관리자가 공유한다. Ansible inventory는 host 그룹당 `ansible_user`를 하나만 가질 수 있어서, `/etc/ansible/inventory.ini` (특정 관리자 계정으로 고정)를 그대로 쓰면 다른 관리자는 그 계정으로 원격 서버에 접속을 시도하다가 SSH 인증에 실패한다. 그래서 관리자마다 자신의 계정으로 접속하는 개인 inventory가 필요하다. (UID\_GID\_Management\_System 5.2.4 내용을 보고 inventory.ini 파일을 `~/uid_gid/inventory.ini`에 위치시킨 경우 지금 설명하는 단계를 진행해야 한다. inventory.ini의 위치가 변경되기 때문이다.)

- 위치: `~/ansible/inventory.ini`. 특정 프로젝트에 속하지 않는 개인 경로다. remote-operations와 `uid_gid` 등 이 데스크탑의 여러 도구가 각자의 설정에서 이 경로를 가리키기만 하고, 어느 한쪽도 이 파일을 소유하지 않는다.
- 준비: `/etc/ansible/inventory.ini`를 복사해서 `ansible_user`만 자신의 계정으로 바꾼다.
- 원격 서버의 `~/.ssh/authorized_keys`에 자신의 공개키를 등록해야 실제 SSH 인증이 된다. inventory 파일만으로는 접속 권한이 생기지 않는다.
- (선택) `~/ansible/ansible.cfg`를 만들어두면 `-i` 없이 `ansible <host> ...` 명령이 기본으로 이 inventory를 사용한다.

```
[defaults]
inventory = /home/<본인계정>/ansible/inventory.ini
interpreter_python = auto_silent
host_key_checking = False
retry_files_enabled = False
```

Ansible 설정이 완료되면 `remote_boot.local.env` 에서 REMOTE\_BOOT\_ANSIBLE\_INVENTORY를 다음과 같이 수정한다.

```
REMOTE_BOOT_ANSIBLE_INVENTORY="/home/<본인계정>/ansible/inventory.ini"
```

이 개인 설정은 dry-run 검증과 수동 운영에 쓰인다. 실제 자동 부팅을 담당하는 `remote-boot.service` (systemd)는 별도 관리자 계정 소유로 이미 배포되어 있으며 이 개인 설정과 무관하게 동작한다.

## MAC 주소

`REMOTE_BOOT_MAC_*` 는 물리 서버 하드웨어의 실제 MAC 주소로 장비통합관리문서에서 값을 가져온다.

## Slack 알림 설정

부팅 실패 알림을 실제로 받아보려면 `remote_boot.local.env` 에서 두 값을 채운다.

```
REMOTE_BOOT_SLACK_ENABLED=true
REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM="https://hooks.slack.com/services/..."
```

- `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 도 MAC 주소와 마찬가지로 실제 운영 값이라 임의로 만들 수 없다. 기존 운영 담당자에게 확인하거나, 이미 배포된 운영 설정에서 값을 가져온다.
- LAB/FARM 구분 없이 모든 원격 부팅 알림은 이 FARM webhook 하나로 전송된다.
- `REMOTE_BOOT_SLACK_NOTIFY_API_URL` 은 webhook을 직접 호출하지 않고 백엔드 내부 notify API를 거친다.
- 설정 후 아래 명령으로 실제 Slack 채널에 메시지가 오는지 확인한다.

```
./test/test_slack_notification.sh --server-id FARM1
```